

Documentación Técnica: API REST con Express

1. ¿Para qué se hizo este proyecto?

El objetivo principal fue construir desde cero un servicio backend basado en la arquitectura cliente-servidor utilizando Node.js y Express. Se implementó una API REST que gestiona recursos mediante el protocolo HTTP, aplicando principios de modularidad, buenas prácticas de desarrollo y control de versiones con Git/GitHub.

2. ¿Cómo se implementó? (Metodología paso a paso)

- Entorno y Control de Versiones: Inicialización del proyecto con Git y npm. Creación de un archivo .gitignore para excluir "node_modules" y mantener un repositorio ligero y profesional.
- Servidor Base y Endpoints: Configuración del servidor en el puerto 3000 con el middleware express.json() para parsear cargas útiles en formato JSON. Implementación del endpoint inicial GET /api/saludo.
- Arquitectura Modular: Separación estricta de responsabilidades dividiendo el proyecto en:
 - Carpeta routes/: Define los verbos y endpoints (/api/estudiantes).
 - Carpeta controllers/: Contiene la lógica de negocio, validaciones y persistencia temporal en memoria.
- Manejo de Errores y Validaciones: En el endpoint POST /api/estudiantes se agregó validación para garantizar que los campos "nombre" y "carrera" existan antes de insertar el registro, respondiendo con código 400 Bad Request en caso de omisión y 201 Created al registrarlo exitosamente.
- Pruebas de Software: Verificación manual mediante navegador para peticiones GET y Postman para la carga útil JSON en peticiones POST.

3. Fundamentos Técnicos Clave

- Arquitectura Cliente-Servidor: Desacoplamiento entre el cliente emisor (Postman/Navegador) y el backend procesador.
- Códigos de Estado HTTP: Uso semántico de respuestas (200 OK para lecturas, 201 Created para creaciones, 400 Bad Request para datos inválidos enviados por el cliente).
- Modelo Asíncrono de Node.js: Manejo no bloqueante de conexiones a través del bucle de eventos (Event Loop).